

IFT436 - Série d'exercices #6 : diviser-pour-régner

Manuel Lafond

Exercice 1: Considérez la récurrence

$$f(n) = \begin{cases} O(1) & \text{si } n = 1 \\ 4f(n/2) + O(n) & \text{sinon} \end{cases}$$

Sans utiliser le théorème maître, montrez que $f(n) \in O(n^2)$.

Indice: si vous essayez de prouver $f(n) \leq cn^2$ par induction, cela va probablement échouer. Essayez de démontrer $f(n) \leq c(n^2 - n)$.

On va montrer que $f(n) \leq c(n^2 - n)$ pour une certaine constante c , par induction. Comme cas de base, on prend $n = 2$. On a $f(2) = 4f(1) + 2c_2 \leq 4c_1 + 2c_2$ pour des constantes c_1, c_2 . On veut que $4c_1 + 2c_2 \leq c(4 - 2)$, ce qui sera vrai pour toute constante $c \geq 2c_1 + c_2$.

Pour l'induction, on suppose que $f(m) \leq c(m^2 - m)$ pour tout $m < n$. Par la définition de f , on a $f(n) \leq 4f(n/2) + c_2n$. En utilisant l'hypothèse d'induction sur $f(n/2)$ et en supposant que n est pair, on obtient $f(n) \leq 4c((n/2)^2 - n/2) + c_2n = cn^2 - 2cn + c_2n = c(n^2 - n) - cn + c_2n$. Notre énoncé sera donc vrai pour $c \geq c_2$. En prenant $c \geq 2c_1 + c_2$, on voit que l'énoncé est correct.

Exercice 2: Considérez l'algorithme artificiel suivant:

```

fonction algoArtificiel( $T$ ) //  $T$  est un tableau de taille  $n$ 
     $A = \text{algoArtificiel}(T[1..n/5])$ 
     $B = \text{algoArtificiel}(T[2n/5..3n/5])$ 
    traitementQuelconque( $A, B$ )
     $C = \text{algoArtificiel}(T[n/5..2n/5])$ 
     $D = \text{algoArtificiel}(T[4n/5..n])$ 
    traitementQuelconque( $C, D$ )
     $E = \text{algoArtificiel}(T[3n/5..4n/5])$ 
     $F = \text{algoArtificiel}(T[1.5n/5..2.5n/5])$ 
     $G = \text{algoArtificiel}(T[2.5n/5..3.5n/5])$ 
    return fonctionQuelconque( $A, B, C, D, E, F, G$ )

```

En supposant que la fonction *traitementQuelconque* s'effectue en temps $O(n)$ et *fonctionQuelconque* en temps $O(1)$, donnez la complexité de cet algorithme.

Et si *traitementQuelconque* s'effectuait en temps $O(n^{1.3})$?

Il suffit d'utiliser le théorème maître. Le problème se divise en $a = 7$ sous-appels sur des instances de taille $n/5$, et donc $b = 5$. Si le traitement est $O(1) = O(n^0)$, on a $c = 0$. Puisque $\log_b a > c$, la complexité est $\Theta(n^{\log_5 7})$.

Si le traitement est $O(n^{1.3})$, il s'avère que $\log_5 7 \leq 1.21$, et ici selon le théorème maître la complexité est $O(n^{1.3})$.

Exercice 3: Considérez le problème du sous-tableau de somme maximum:

Entrée: un tableau $T = [t_1, \dots, t_n]$ d'entiers positifs ou négatifs.

Sortie: un sous-tableau contigu $T' = [t_i, t_{i+1}, \dots, t_{i+k}]$ tel que la somme $\sum_{j=i}^{i+k} T[j]$ est maximum parmi tous les choix possibles.

Donnez un algorithme en temps $O(n)$ pour ce problème.

Indice: il y a plusieurs façons d'y arriver. En particulier, il y a une méthode diviser-pour-régner. Pour combiner les sous-solutions, les sous-appels, en plus du sous-tableau maximum, pourraient retourner d'autres informations.

Notre algorithme va retourner plus de valeurs qu'il n'en faut. Il nous faut quelques définitions.

- a. On appelle la *somme maximum* la somme des éléments d'un sous-tableau contigu de valeur maximum (donc, ce qu'on veut obtenir à la fin).
- b. On appelle la *somme maximum préfixe* la somme maximum possible d'un sous-tableau contigu de T tel que $T[1]$ est dans ce sous-tableau (donc la meilleure somme possible en commençant du début).
- c. On appelle la *somme maximum suffixe* la somme maximum possible d'un sous-tableau contigu de T tel que $T[n]$ est dans ce sous-tableau (donc la meilleure somme possible en commençant par la fin).
- d. On appelle la *somme totale* la somme de tous les éléments de T .

Mis à part la somme totale, toutes ces valeurs pourraient être à 0 si on décide de n'inclure aucun élément. Nous n'allons pas nous attarder à trouver les indices des sous-tableaux - seuls les valeurs vont nous intéresser, mais l'algorithme pourrait être modifié facilement.

L'algorithme retourne (m, d, f, t) , où m est la somme maximum, d la somme maximum préfixe, f la somme maximum suffixe et t la somme totale. Je recommande de vous faire un dessin pour bien comprendre comment chacune des valeurs est calculée.

```

fonction sousTabMax( $T$ )
  si  $|T| = 1$  alors
    si  $T[1] < 0$  alors
      return  $(0, 0, 0, T[1])$ 
    sinon
      return  $(T[1], T[1], T[1], T[1])$ 
   $(m_1, d_1, f_1, t_1) = \text{sousTabMax}([t_1, \dots, t_{n/2}])$ 
   $(m_2, d_2, f_2, t_2) = \text{sousTabMax}([t_{n/2+1}, \dots, t_n])$ 
  // On calcule maintenant  $(m, d, f, t)$  à retourner
   $t = t_1 + t_2$ 
   $d = \max(d_1, t_1 + d_2)$ 
   $f = \max(f_2, f_1 + t_2)$ 
   $m = \max(m_1, m_2, f_1 + d_2)$ 
  return  $(m, d, f, t)$ 

```

La récurrence pour cet algorithme donne $f(n) = 2f(n/2) + O(1)$. Selon le théorème maître, ceci est $O(n)$.

Exercice 4: Considérez le problème de calculer x^y donc x à la puissance y , où x et y sont des entiers donnés.

L'algorithme naïf fait la chose suivante:

Soit $r = x$.

Pour $i = 1$ à y , faire $r = r \cdot x$.

Retourner r .

Si on suppose que les multiplications se font en temps constant, la complexité de cet algorithme est $O(y)$. Donnez un algorithme qui effectue la même tâche, mais en temps $O(\log y)$.

Il suffit de réduire l'exponentiation de moitié et de multiplier.

```
fonction pow( $x, y$ )  
  si  $y$  est pair alors  
     $p = \text{pow}(x, y/2)$   
    return  $p * p$   
  sinon  
     $p = \text{pow}(x, (y - 1)/2)$   
    return  $p * p * x$ 
```

Exercice 5: Considérez l'algorithme diviser-pour-régner pour le problème des deux points les plus proches 2D. Sa complexité était de $O(n \log^2 n)$. Montrez comment améliorer cette complexité à $O(n \log n)$.

Note: l'indice est dans les notes de cours.

La partie qui prenait le plus de temps dans l'algorithme était de trier après chaque retour des appels récursifs. On peut plutôt demander aux appels récursifs de retourner un tableau des points déjà triés, puis faire une fusion.

```
fonction pointsPlusProches2D( $P = [p_1, \dots, p_n]$ )
  //On suppose que  $P$  est déjà trié selon les coordonnées en  $x$ 
  //Ceci peut être fait par l'appel initial et coûte  $O(n \log n)$ 
  une seule fois.
  //On va aussi supposer que  $n$  est pair, sinon on ajoute un
  point bidon.
  //Et on suppose que pointsPlusProches2D retourne les
  points  $r_1, r_2$  les plus proches, et les points triés en  $y$ .
   $(r_1, r_2, P_1) = \text{pointsPlusProches2D}([p_1, \dots, p_{n/2}])$ 
   $(q_1, q_2, P_2) = \text{pointsPlusProches2D}([p_{n/2+1}, \dots, p_n])$ 
   $d = \min(\text{dist}(r_1, r_2), \text{dist}(q_1, q_2))$ 
   $R = \text{fusion}(P_1, P_2)$  // Trie  $P_1, P_2$  selon les  $y$  en
  fusionnant, temps  $O(n)$ 
   $R' =$  points de  $R$  à distance  $\leq d$  de la ligne entre  $p_{n/2}, p_{n/2+1}$ 
  pour  $i = 1..|R'|$  faire
    Comparer  $R'[i]$  avec  $R'[i - 32]$  à  $R'[i + 32]$ , mettre  $d$  à
    jour si on trouve des points plus proches
  fin
  Soit  $s_1, s_2$  les points les plus proches trouvés
  return  $(s_1, s_2, R)$ 
```

Ce qui a changé par rapport à avant, c'est que le traitement pour combiner des solutions se fait en temps $O(n)$. La récurrence est $f(n) = 2f(n/2) + O(n)$, ce qui est $O(n \log n)$.

Exercice 6: Dans le problème de la **majorité**, on reçoit un tableau T d'objets quelconques et on veut savoir. On suppose que $|T| = n$ est pair, et on veut savoir s'il existe un élément x qui est présente au moins $n/2 + 1$ fois dans T .

- a. Si vous pouvez trier les éléments de T , i.e. $x < y$ est bien défini sur les objets de T , donnez un algorithme en temps $O(n \log n)$ pour le problème de la majorité.

On trie, puis on parcourt de gauche à droite en comptant le nombre d'occurrences de l'élément courant. On garde en mémoire l'élément qui apparaît le maximum de fois, et on retourne *true* s'il est présent $n/2 + 1$ fois ou plus.

- b. Supposons maintenant que $x < y$ n'est pas défini, i.e. vous avez des objets incomparables. Par contre, vous pouvez vérifier l'égalité, donc pour deux éléments de T , vous avez accès à $x = y$ qui retourne *true* si x et y sont égaux, et *false* sinon.

Donnez un algorithme en temps $O(n \log n)$ pour le problème de la majorité. Considérez une méthode diviser-pour-régner.

L'observation principale est que si m est un élément majoritaire, alors il doit être majoritaire dans le tableau gauche ou bien le tableau droit (ou les deux).

```

fonction majo( $T = [t_1, \dots, t_n]$ )
  //Retourne un élément majoritaire, ou nil s'il n'y en a
  pas
   $m_1 = \text{majo}([t_1, \dots, t_{n/2}])$ 
   $m_2 = \text{majo}([t_{n/2+1}, \dots, t_n])$ 
  si  $m_1 \neq \text{nil}$  alors
     $c_1 = \text{nombre d'occurrences de } m_1 \text{ dans } T$ 
    si  $c_1 > n/2$  alors
      return  $m_1$ 
  si  $m_2 \neq \text{nil}$  alors
     $c_2 = \text{nombre d'occurrences de } m_2 \text{ dans } T$ 
    si  $c_2 > n/2$  alors
      return  $m_2$ 
  return nil

```

La récurrence pour cet algorithme est $f(n) = 2f(n/2) + O(n)$, ce qui est $O(n \log n)$.

- c. (BONI, PLUS DIFFICILE) Donnez un algorithme en temps $O(n)$ qui vérifie si T contient un élément majoritaire. La méthode diviser-pour-régner ne fonctionne peut-être pas ici.

La réponse facile: avec une table de hachage H , vous pouvez traverser le tableau de gauche à droite et faire $H[T[i]]+ = 1$ pour chaque élément $T[i]$. Vous comptez ainsi le nombre d'occurrence de chaque élément en temps $O(n)$ avec forte probabilité, et il vous suffit de retourner true si un élément apparaît $n/2 + 1$ fois ou plus. On peut faire mieux, mais c'est long à expliquer! J'invite les intéressés à regarder l'algorithme de majorité de Boyer-Moore: https://en.wikipedia.org/wiki/Boyer%E2%80%93Moore_majority_vote_algorithm. Expliqué en termes simples ici: <http://codinghelmet.com/exercises/majority-element>.

Exercice 7: Vous avez un algorithme récursif dont l'analyse en temps donne la récurrence suivante:

$$f(n) = \begin{cases} 1 & \text{si } n = 0, 1 \\ 2f(n-1) + 8f(n-2) + O(1) & \text{sinon} \end{cases}$$

Donnez la valeur de f en terme de complexité O .

On va commencer par se convaincre que le $O(1)$ ne change pas la complexité. Combien de fois ce terme constant $O(1)$ sera-t-il ajouté? Si on représente l'arbre de la récurrence de $f(n)$, ce $O(1)$ sera présent autant de fois qu'il y a de sommets internes dans cet arbre. Soit $t(n)$ le nombre de sommets dans cet arbre de récurrences, au total. La racine représente l'appel $f(n)$, et il y a 10 enfants: deux représentent un appel à $f(n-1)$, et 8 enfants un appel à $f(n-2)$. Le nombre de sommets est donc $t(n) = 1 + 2t(n-1) + 8t(n-2) \leq 2t(n-1) + 8t(n-2) + O(1)$. Donc $t(n) \leq f(n)$. Ceci veut dire que la complexité de l'algorithme est bornée par $O(1) \cdot f(n)$, et donc il suffit de calculer $f(n)$.

On applique la recette vue en classe. On pose $f(n) = 2f(n-1) + 8f(n-2)$. On devine que $f(n) = r^n$ et on a

$$r^n = 2r^{n-1} + 8r^{n-2}$$

Ceci mène à

$$r^2 = 2r + 8$$

puis à

$$r^2 - 2r - 8 = 0$$

On peut factoriser en

$$(r + 2)(r - 4) = 0$$

et les racines sont donc 4 et -2 . Donc toutes les solutions ont la forme $f(n) = c_1 4^n + c_2 (-2)^n$. On trouve maintenant c_1 et c_2 .

Avec $f(0) = 1 = c_1 + c_2$, on a $c_2 = 1 - c_1$.

Ensuite,

$$f(1) = 1 = 4c_1 + (1 - c_1)(-2) = 4c_1 - 2 + 2c_1 = 6c_1 - 2$$

Donc

$$3 = 6c_1$$

Donc $c_1 = 1/2$ et $c_2 = 1/2$. Au final,

$$f(n) = 1/2 4^n + 1/2 (-2)^n$$